

Laptop Configuration & Pricing Management System

Full-Stack Engineering Assignment Documentation

Candidate Role: Full-Stack Engineer (Fresher)

Technology Stack: MERN (MongoDB, Express.js, React.js, Node.js)

System Name: OmniConfig Pricing & Quotation Engine

1. Executive Summary & Problem Overview

Electronics retailers handle thousands of customizable laptop permutations across multiple dynamic hardware categories (CPU, RAM, NVMe Storage, GPU, Displays, Batteries, Keyboards, Operating Systems).

Prior Pain Points (Spreadsheet-based workflow):

- **Pricing Out-of-Sync:** Supplier price updates caused accidental recalculation of historical quotations, damaging customer trust.
- **Manual Errors:** Human miscalculations in margin profit, taxes, and volume discounts led to financial losses.
- **Duplicate Configurations & Slow Response:** Sales executives spent 20-30 minutes manually looking up component SKUs and creating PDF quotes.

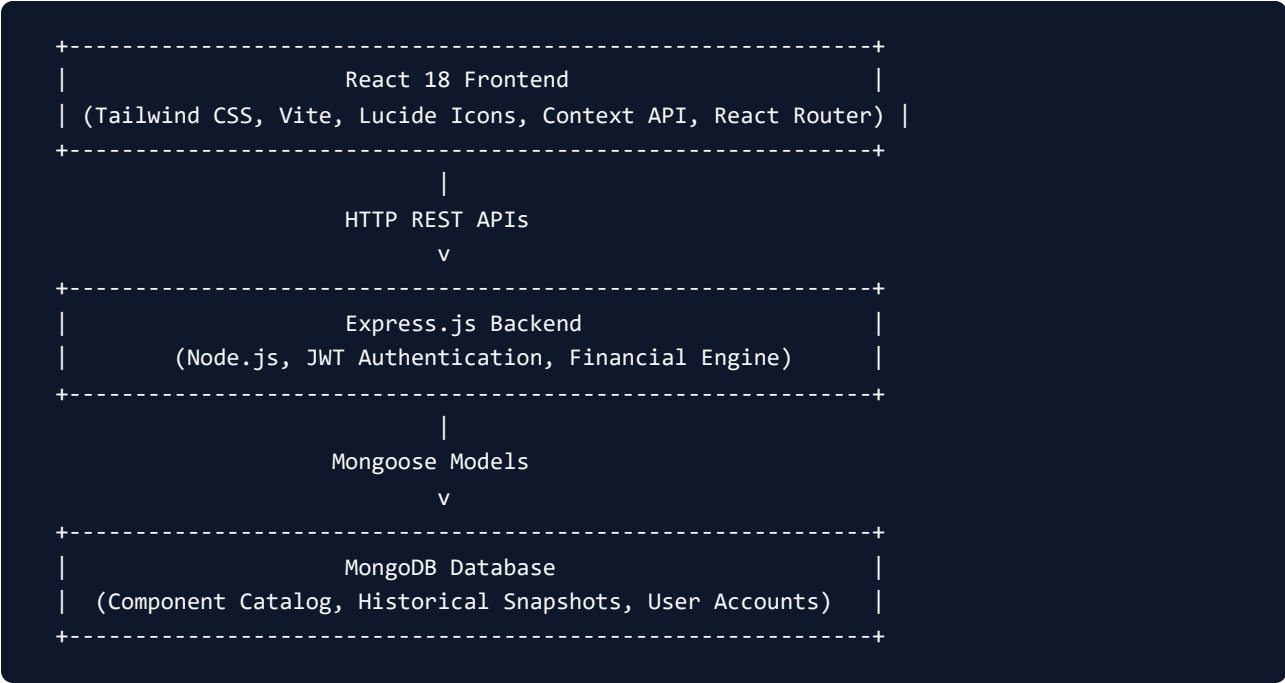
Technical Solution Delivered:

A modern, web-based Full-Stack MERN application with:

- 1. **Automated Real-Time Pricing Calculator:** Instantly computes base costs, selling totals, tax rates, volume discounts, and profit margins.
- 2. **Historical Price Preservation System:** Uses component snapshot data patterns. Saved quotations lock component pricing at quote creation time, guaranteeing zero silent price mutations when master catalog prices change.
- 3. **Hardware Catalog Management (CRUD):** Full control over hardware specifications, stock levels, and price history tracking.
- 4. **Search & Multi-Filtering Engine:** Filter quotations and components by price range, customer name, quote status, or SKU code.
- 5. **Printable / Exportable Invoices:** One-click official quotation document generation.

2. System Architecture & Database Schema Design

High-Level Architecture



Database Models

A. Master Component Schema (`Component.js`)

```
{
  sku: { type: String, required: true, unique: true },
  name: { type: String, required: true },
  category: {
    type: String,
    enum: ['Processor', 'RAM', 'Storage', 'Graphics Card', 'Display', 'Battery', 'Keyboard', 'Op
  ],
  brand: String,
  specifications: Map, // Dynamic Key-Value pairs e.g. { Cores: "14", Clock: "5.0 GHz" }
  baseCost: Number,    // Supplier purchase cost
  sellingPrice: Number, // Retail price
  stockQuantity: Number,
  isAvailable: Boolean,
  priceHistory: [
    { sellingPrice: Number, baseCost: Number, updatedAt: Date, updatedBy: String, reason: String
  ]
}
```

B. Quotation Schema & Historical Snapshot Pattern (`Quotation.js`)

```
{
  quoteNumber: { type: String, unique: true }, // e.g. QUO-20260807-0001
  configName: String,
  customerName: String,
  customerEmail: String,
  customerPhone: String,

  // IMMUTABLE PRICE SNAPSHOT (Solves Historical Preservation requirement)
  components: [
    {
      componentId: ObjectId,
      sku: String,
      name: String,
      category: String,
      sellingPriceAtQuote: Number, // Snapshot price at creation moment
      baseCostAtQuote: Number,
      specifications: Map
    }
  ]
}
```

```
],  
  
pricingSummary: {  
  componentsSubtotalCost: Number,  
  componentsSubtotalSelling: Number,  
  discountPercentage: Number,  
  discountAmount: Number,  
  taxPercentage: Number,  
  taxAmount: Number,  
  finalTotal: Number,  
  marginAmount: Number,  
  marginPercentage: Number  
},  
  
status: { type: String, enum: ['Draft', 'Quoted', 'Approved', 'Rejected', 'Fulfilled'] },  
createdByName: String  
}
```

3. Key Technical Decisions & Design Rationale

1. Why Snapshot Objects for Historical Preservation?

- *Alternative Considered:* Referencing Component `_id` directly without snapshots.
- *Drawback:* If supplier prices rise by 15% next month, fetching an old quotation by ID would dynamically recompute higher prices, invalidating previously issued formal customer quotes!
- *Our Solution:* Storing `sellingPriceAtQuote` inside the quotation array ensures absolute pricing immutability while allowing comparison against current market prices.

2. React Context API for State Management:

- Lightweight, scalable, zero setup overhead compared to Redux Toolkit for this application scope.
- Separate contexts for `AuthContext` and `ToastContext` keep component re-renders optimized.

3. Tailwind CSS + Glassmorphism Aesthetics:

- High-contrast Slate dark theme (`#090d16`), cyan glassmorphic panels, glowing badge highlights, and custom print layout support (`@media print`).

4. API Endpoints Documentation

Method	Endpoint	Description
POST	<code>/api/auth/login</code>	Authenticate user & return JWT token
GET	<code>/api/components</code>	List catalog components (supports <code>?category=</code> & <code>?search=</code>)
POST	<code>/api/components</code>	Create new hardware component
PUT	<code>/api/components/:id</code>	Update component details
PATCH	<code>/api/components/:id/price</code>	Quick price update & log to price history
DELETE	<code>/api/components/:id</code>	Remove component from catalog
GET	<code>/api/quotations</code>	List quotations (supports <code>?status=</code> , <code>?search=</code> , <code>?minPrice=</code> , <code>?maxPrice=</code>)
GET	<code>/api/quotations/:id</code>	Detailed quotation view with snapshot verification
POST	<code>/api/quotations</code>	Generate new quotation with snapshot calculation
PUT	<code>/api/quotations/:id/status</code>	Update quote lifecycle status
GET	<code>/api/analytics/dashboard</code>	Fetch KPI analytics & distribution stats
POST	<code>/api/seed</code>	Reset/Seed database with realistic initial hardware catalog

5. Local Setup & Execution Guide

Prerequisites

- Node.js (v18 or higher)
- MongoDB instance (Local or MongoDB Atlas)

Step 1: Start Backend Server

```
cd backend  
npm install  
npm start
```

Backend runs on `http://localhost:5000` and automatically seeds initial hardware components and demo accounts if empty.

Step 2: Start Frontend Application

```
cd frontend  
npm install  
npm run dev
```

Frontend runs on `http://localhost:3000` with preconfigured API proxying.

Evaluator Demo Credentials:

- **Primary Pricing Manager:** `prashantdasar2004@gmail.com` / `Pachhi@123`
- **Sales Executive:** `sales@retailer.com` / `password123`

6. Video Recording Presentation Script (8–12 Minutes Guide)

Part 1: Introduction & Problem Context (1.5 Mins)

- Introduce yourself and the assignment goal: building a Laptop Configuration & Pricing Management System for electronics retailers.
- Highlight the core business problem: spreadsheets caused wrong quotes, missing margin visibility, and lost pricing history.

Part 2: Interactive Laptop Builder & Real-Time Calculation (3 Mins)

- Demo navigating to the **Laptop Builder** page.
- Select hardware components across categories (Intel i7, 32GB DDR5 RAM, RTX 4070 GPU, 4K Display, Windows 11 Pro).
- Show the **Sticky Live Calculator** sidebar updating subtotal, gross margin profit (\$ and %), discount slider, and sales tax in real-time.
- Enter customer details (Acme Corp) and click **Confirm & Save Quotation**.

Part 3: Historical Pricing Preservation Proof (3 Mins)

- Open the newly generated quotation (e.g., total price \$2,150).
- Open the **Components Catalog** in another tab. Select the Intel i7 processor and use the **Update Price** modal to increase its price by +\$100.
- Return to the saved quotation page: demonstrate that the quotation total remains locked at \$2,150, while the page highlights the "Current Catalog Delta" (+ \$100).
- Explain how the `sellingPriceAtQuote` snapshot pattern protects business integrity.

Part 4: Component Catalog & Search/Filtering (2 Mins)

- Show the Components page: filtering by categories, adding a new hardware spec, viewing the price revision audit trail.
- Show the Saved Quotations page: filtering by price range (\$1,000 - \$3,000) and searching by customer email.

Part 5: Print Invoice & Architecture Wrap-Up (1.5 Mins)

- Click **Print Official Invoice** on any quotation to show printable layout.
- Summarize MERN architecture, schema design, and clean separation of concerns.